



HACKTHEBOX

Einladen Writeup



Prepared by: Sebh24

Machine Author(s): thewhitefriday

Difficulty: **Medium**

Scenario

Our staff recently received an invite to the German embassy to bid farewell to the Germany Ambassador. We believe this invite was a phishing email due to alerts that fired on our organisation's SIEM tooling following the receipt of such mail. We have provided a wide variety of artifacts inclusive of numerous binaries, a network capture, DLLs from the host system and also a .hta file. Please analyse and complete the questions detailed below!

Analysis

As usual, we download the relevant zip file, unzip and then confirm the contents:

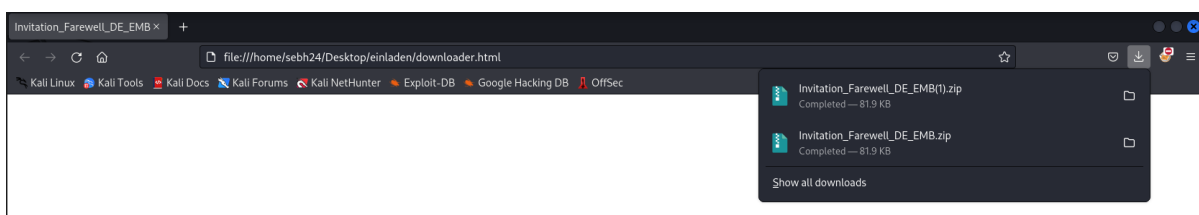
```
└─(sebh24@maroc)-[~/Desktop/einladen]
└─$ ls -alh
total 19M
drwxr-xr-x  2 sebh24 sebh24 4.0K Apr  4 16:56 .
drwxr-xr-x  8 sebh24 sebh24 4.0K Apr  4 16:56 ..
-rw-r--r--  1 sebh24 sebh24 2.5K Jul 28  2023 AppVIsvSubsystems64.dll
```

```

-rw-r--r-- 1 sebh24 sebh24 294K Aug 12 2023 downloader.html
-rw-r--r-- 1 sebh24 sebh24 62K Aug 30 2023 EmpireClient.exe
-rw-r--r-- 1 sebh24 sebh24 295K Jul 28 2023 Invitation_Farewell_DE_EMB.hta
-rw-r--r-- 1 sebh24 sebh24 82K Aug 30 2023 Invitation_Farewell_DE_EMB.zip
-rw-r--r-- 1 sebh24 sebh24 25K Jul 28 2023 Invitation.pdf
-rw-r--r-- 1 sebh24 sebh24 15M Aug 16 2023 Logfile.PML
-rw-r--r-- 1 sebh24 sebh24 33K Jul 28 2023 mso.dll
-rw-r--r-- 1 sebh24 sebh24 59K Jul 28 2023 msoev.exe
-rw-r--r-- 1 sebh24 sebh24 53K Aug 16 2023 msoev.pcapng
-rw-r--r-- 1 sebh24 sebh24 2.7M Aug 9 2023 sheet.hta
-rw-r--r-- 1 sebh24 sebh24 174K Aug 30 2023 unc.js

```

We are provided with a brief of the incident which highlights an invite to the German embassy. Taking this into account we initially open the HTML file using the web browser of our choice. This will ask us to download a the zip file, and following the download we take a SHA256 hash of the zip file.



```

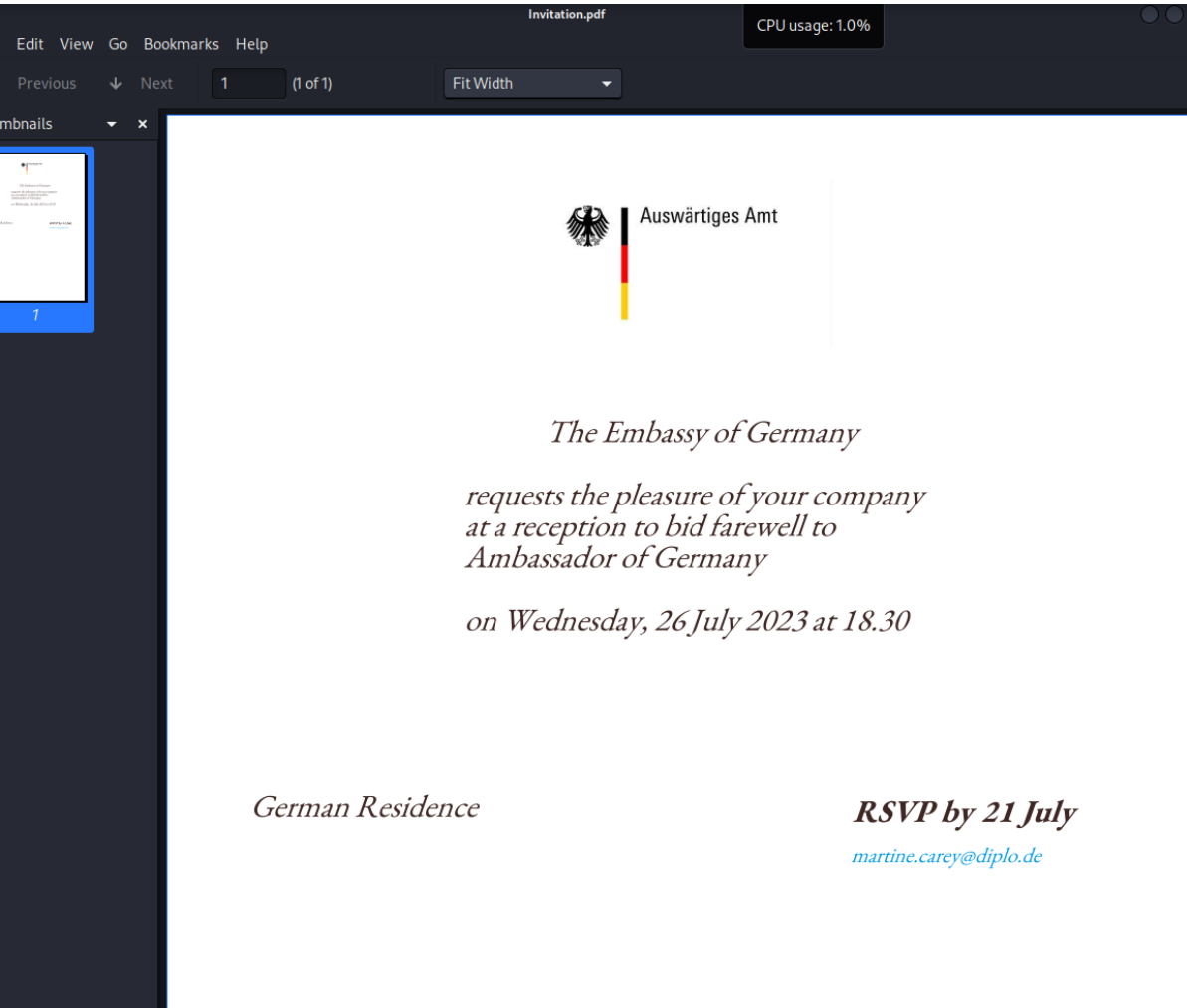
(sebh24@maroc) - [~/Desktop/einladen]
└─$ sha256sum Invitation_Farewell_DE_EMB.zip
5d4bf026fad40979541efd2419ec0b042c8cf83bc1a61cbcc069efe0069ccd27
Invitation_Farewell_DE_EMB.zip

```

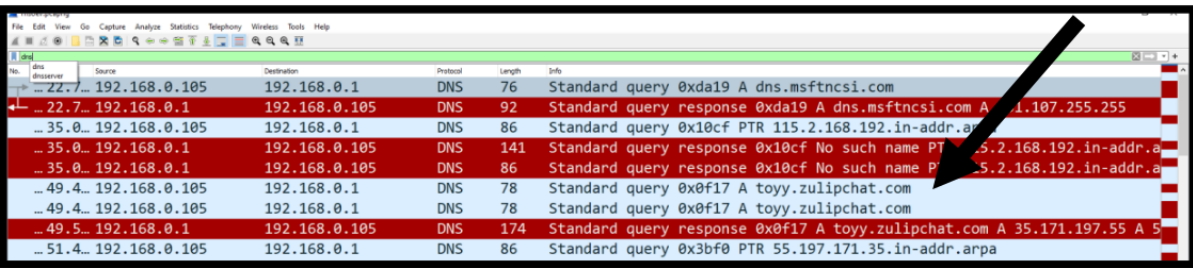
The ZIP file acquired during the investigation harbors an HTA document, notably titled 'Invitation_Farewell_DE_EMB.hta'. Upon inspection of this HTA file, it's observed that it ingeniously employs a set of variables to encapsulate the binary data for several file types, including executables (EXE), Dynamic Link Libraries (DLL), and Portable Document Format (PDF) files. Intriguingly, towards the culmination of the script, precisely at the 79th line, an execution command is employed to initiate the EXE file that has been dynamically generated through the script's operations. The command in question, `CreateObject("WScript.Shell").Exec "C:\\windows\\tasks\\msoev.exe"`, serves to execute the `msoev.exe` file directly from the Windows system's tasks directory.

To delve deeper into the analysis while circumventing the automatic execution of the potentially malicious executable, this particular line can be strategically removed from the HTA file. This manipulation allows for the execution of the HTA script in a controlled environment, facilitating the extraction of the EXE, DLL, and PDF files without initiating the malicious EXE file. Following the extraction, a meticulous examination of the file properties was conducted. A right-click on each file, followed by a navigation to the Properties dialogue, enabled a review of their digital certification status. This review unveiled that exclusively the EXE file boasted a 'Digital Signatures' tab within its Properties. Notably, the 'Name of signer' listed under this tab was none other than Microsoft Corporation, a detail that could potentially mislead an observer regarding the file's legitimacy. It's important to highlight that the creation of the EXE file, as facilitated by the HTA script, was directly attributed to the data specified within the 'msoev' variable. This discovery underscores the sophisticated nature of the attack, leveraging the guise of legitimacy through digital signatures to mask the deployment of malicious software.

Upon opening the PDF file "Invitation.pdf", generated by the HTA file we see that the threat actor pretending to be a contact at the German embassy.



Upon analysis of the network traffic captured in the PCAP file using Wireshark, an intriguing pattern emerges when filtering for DNS requests. The malware initiates communication attempts with a domain identified as `toyy.zulipchat.com`. This domain is associated with Zulip, a known platform for chat solutions, which, under ordinary circumstances, serves as a legitimate tool for team collaboration and communication. However, the utilisation of such a platform by the malware for communication purposes is highly unconventional and warrants a deeper examination.



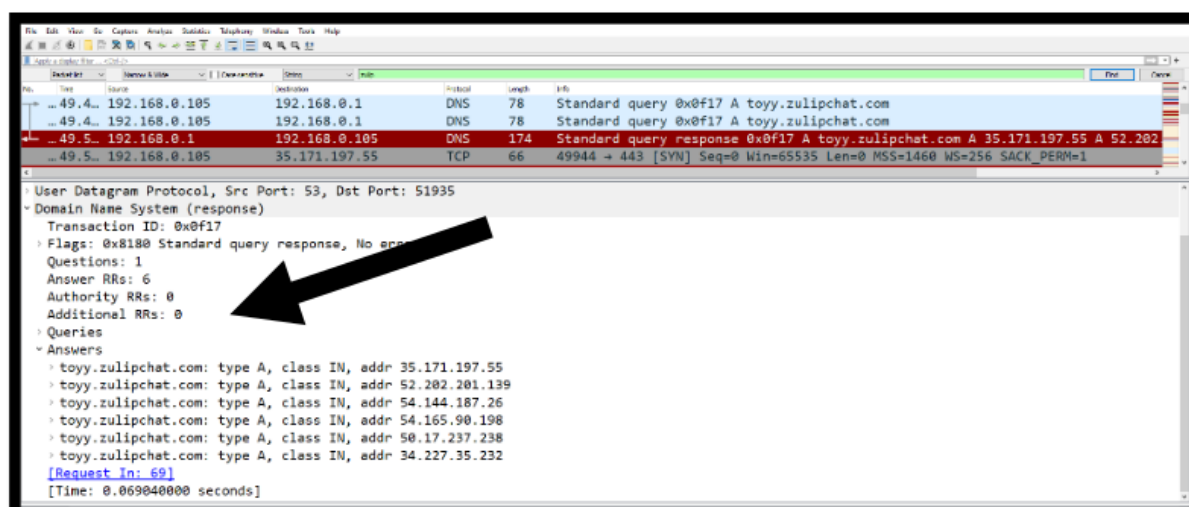
This stems from the malware's exploitation of a reputable service for potentially malicious activities. Malware traditionally communicates with domains or IP addresses controlled by threat actors, often specifically set up for command and control (C&C) purposes. These domains are usually obscure or mimic legitimate websites to evade detection. However, in this instance, the malware's choice of `toyy.zulipchat.com` as a communication endpoint deviates from typical patterns, leveraging the inherent trust and legitimacy of an established chat service platform.

The specific IP address mentioned, `35.171.197.55`, further complicates the analysis. This IP address is resolved from the DNS query to `toyy.zulipchat.com`, indicating that the malware's communication is not just with any arbitrary server but with one that is potentially housed within a reputable cloud service provider's infrastructure. This could imply a higher level of sophistication in the attack, as leveraging cloud services can obfuscate the malware's network traffic within legitimate web traffic, making detection and mitigation efforts more challenging.

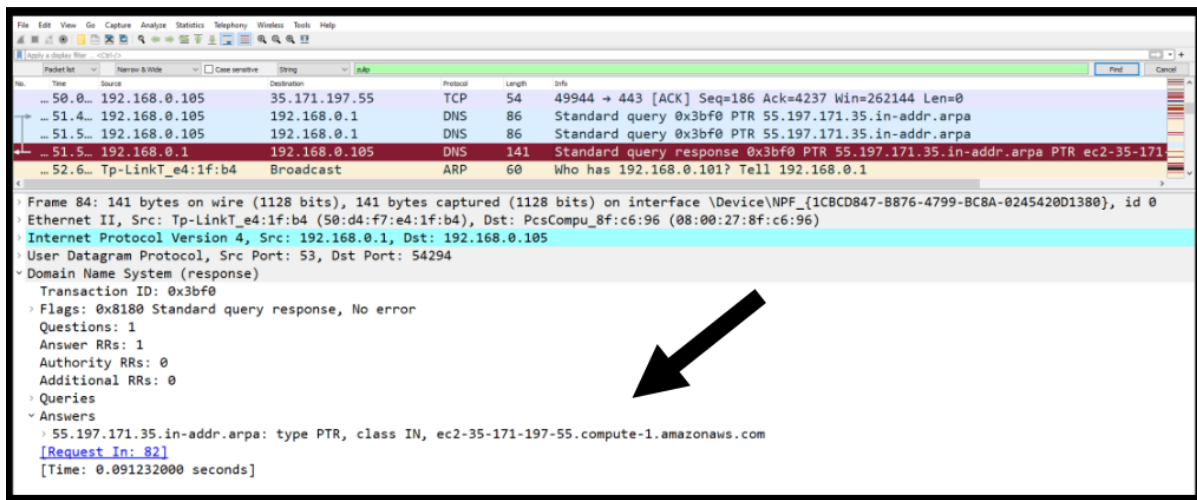
This unusual method of communication could serve multiple purposes for the malware:

1. **Evasion:** By communicating through a legitimate and widely used service, the malware attempts to blend in with normal, benign traffic, reducing the likelihood of detection by network monitoring tools that are tuned to flag known malicious domains.
2. **Reliability:** Using a stable and reliable platform like Zulip ensures that the malware's C&C communications are less likely to be interrupted compared to using a dedicated malicious server, which might be taken down by service providers once identified.
3. **Anonymity:** Leveraging cloud-based services complicates the attribution process, making it harder for responders to trace the malware's control back to the attackers, as it hides among the multitude of legitimate users.

Upon closer examination of the DNS response, specifically within the 'Domain Name System (Response)' section, a notable detail emerges under the 'Answers' subsection. Here, it is discovered that a total of six DNS A records are returned for the Zulip chat domain in question.



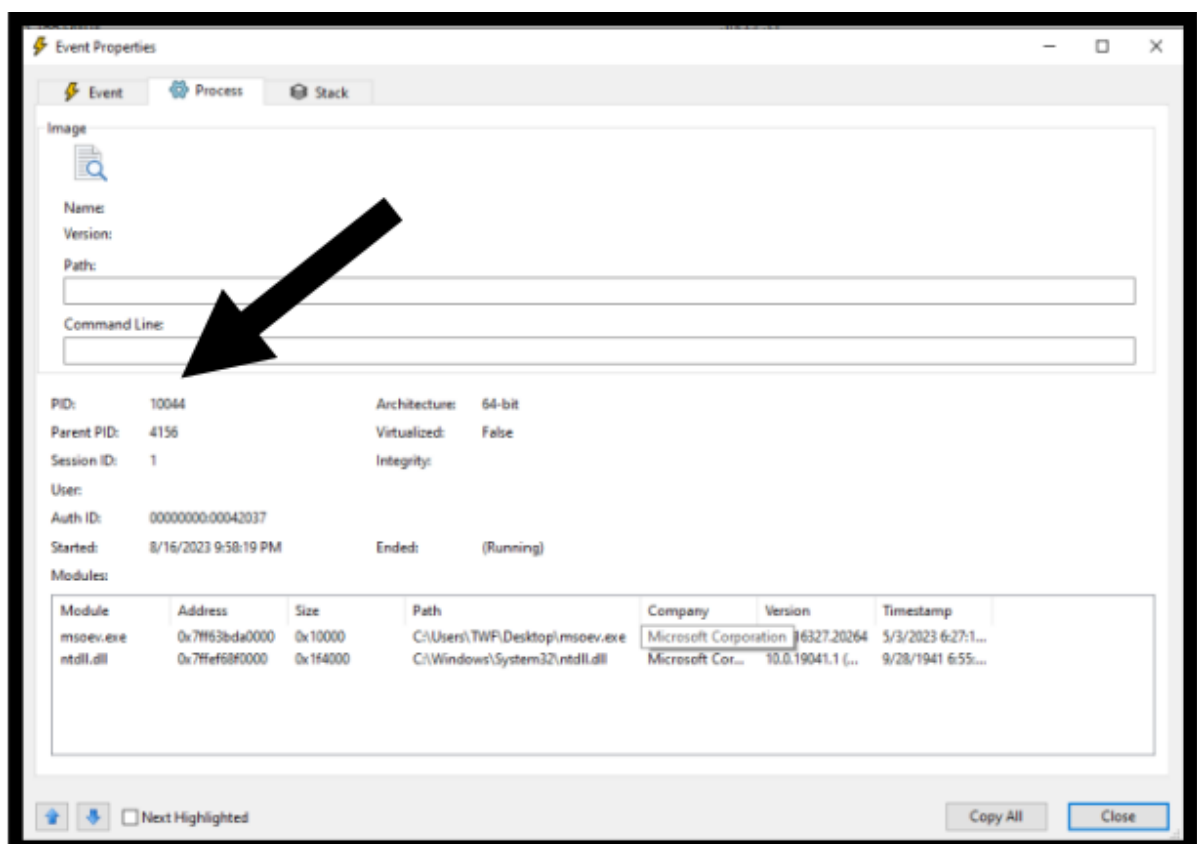
When delving deeper into the traffic, beyond the initial exchanges, an IP address associated with the Zulip chat domain's DNS response emerges as a point of interest. This IP address is identified as belonging to an Amazon Web Services (AWS) Elastic Compute Cloud (EC2) instance - `ec2-35-171-197-55.compute-1.amazonaws.com`



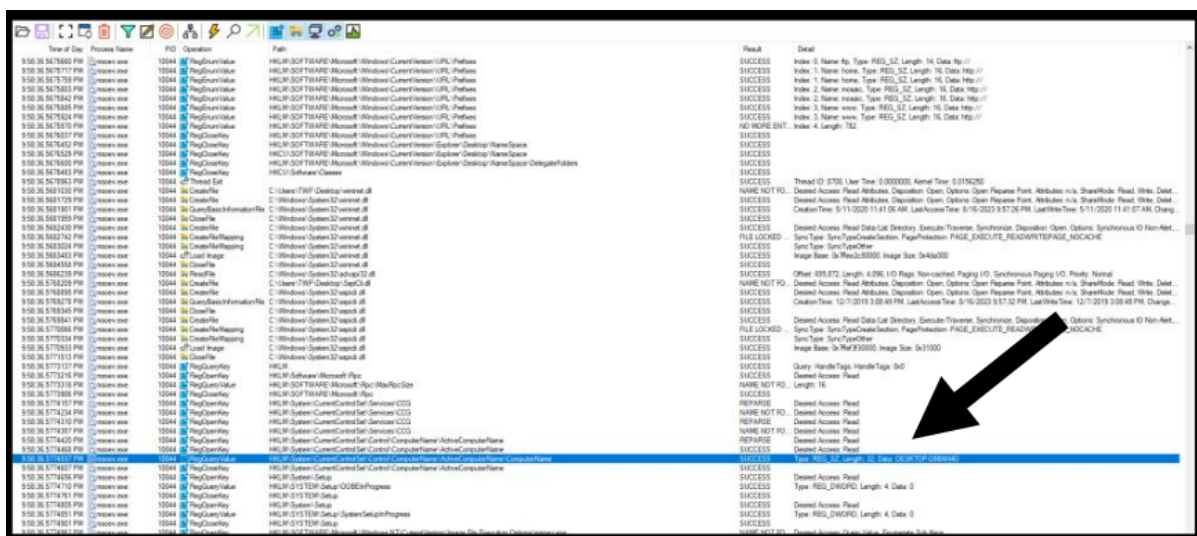
This observation is notable for several reasons:

1. **Cloud-Based Infrastructure:** The utilization of an AWS EC2 instance as part of the malware's command and control (C&C) infrastructure is indicative of a trend where attackers leverage reputable cloud services. This approach offers reliability and scalability for their operations, benefiting from the robust infrastructure provided by cloud platforms.
2. **Evasion and Anonymity:** The choice of AWS, a widely used and trusted cloud service, as a hosting platform for the C&C server, complicates detection efforts. Traffic to and from AWS servers is not uncommon, making it more challenging for network security tools to distinguish between legitimate and malicious communications without detailed analysis.
3. **Operational Resilience:** Hosting the C&C server on a cloud platform like AWS provides operational resilience. Cloud services typically offer high uptime and redundancy, ensuring that the C&C server remains accessible to the malware even in the face of takedown attempts or network fluctuations.

Lets delve into detail around the mosev.exe now. Upon opening the PML file using the "Process Monitor" tool, we navigate to the Process tab for the malware process msoev.exe and confirm the PID & PPID of the process. 4156 & 10044 respectively.



We are able to confirm the computer name of the compromised asset by searching for "ComputerName" in Process Monitor and locating the value within the Registry key `HKLM\System\CurrentControlSet\Control\ComputerName\ActiveComputerName\ComputerName`.



Our analysis concludes the compromised host name as `DESKTOP-088AN40`.

Using a similar process, except modifying our search for `Users` we are able to also locate the name of the user within Process Monitor, TWF.



The malware sets 11 data values within the Windows Registry on the compromised machine, which we are able to confirm by filtering for events with `Operation is RegSetValue`.

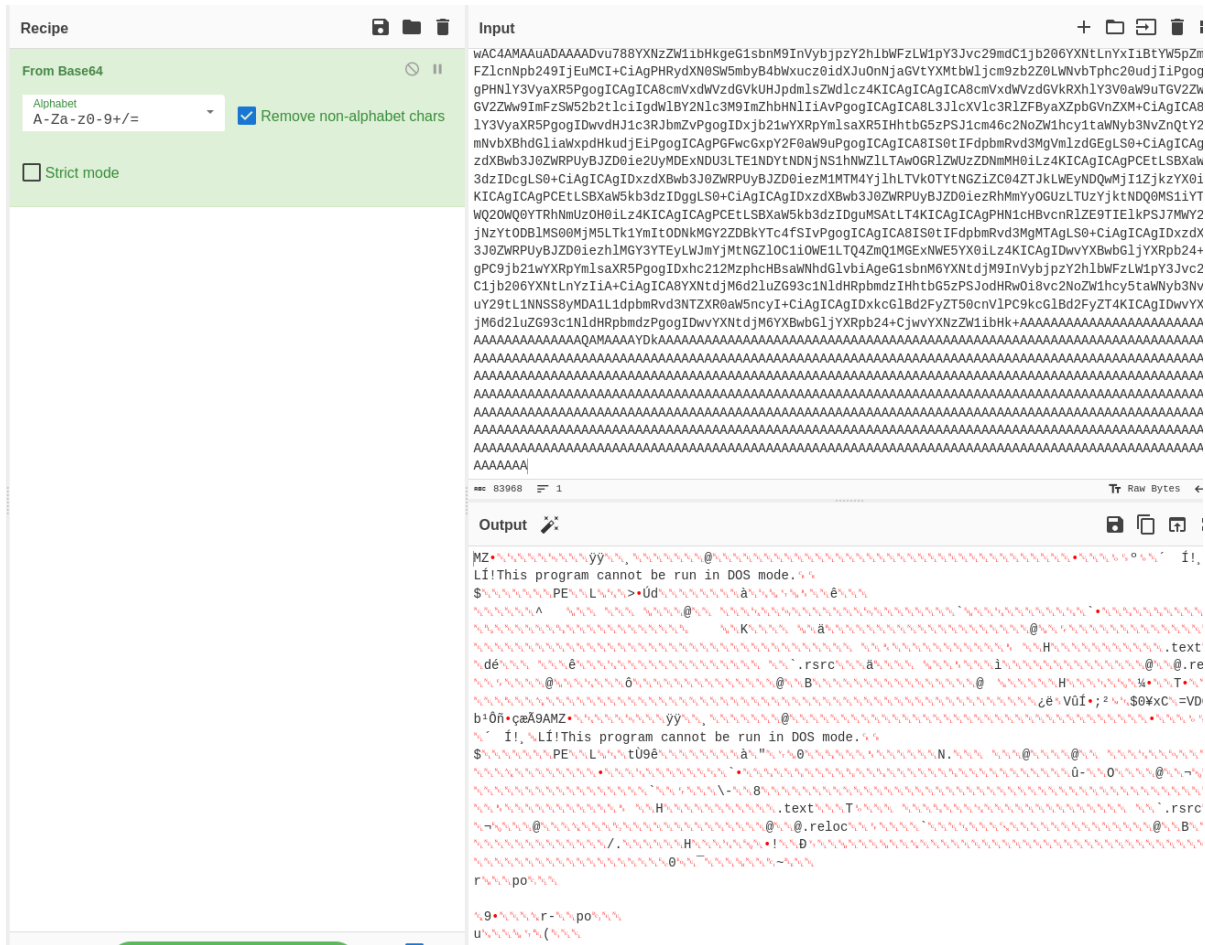
And finally within Process Monitor we are able to navigate to stack > Process tab > Modules List and confirm the existence of mso.dll, indicating it was successfully loaded.

Upon submitting the hash to Virustotal - <https://www.virustotal.com/gui/file/1754dd14b25884fb4c7fc1663cb56a8c169efdd59e173aa1573cd24eda21c393/behavior>, we are able to locate numerous dropped files. Of particular note is richpear.bat.

Files Written

-  C:\Users\ADMINI~1\AppData\Local\Temp\jumpyflame
-  C:\Users\ADMINI~1\AppData\Local\Temp\richpear.bat
-  C:\Users\ADMINI~1\AppData\Local\Temp\rosecomb.dll
-  C:\Users\user\AppData\Local\Microsoft\Windows\Caches
-  C:\Users\user\AppData\Local\Temp\jumpyflame
-  C:\Users\user\AppData\Local\Temp\richpear.bat
-  C:\Users\user\AppData\Local\Temp\richpear.bat:Zone.Identifier
-  C:\Users\user\AppData\Local\Temp\richpear.bat\Zone.Identifier:\$DATA
-  C:\Users\user\AppData\Local\Temp\rosecomb.dll
-  C:\Windows\cer45D3.tmp
-  \Device\ConDrv\Connect

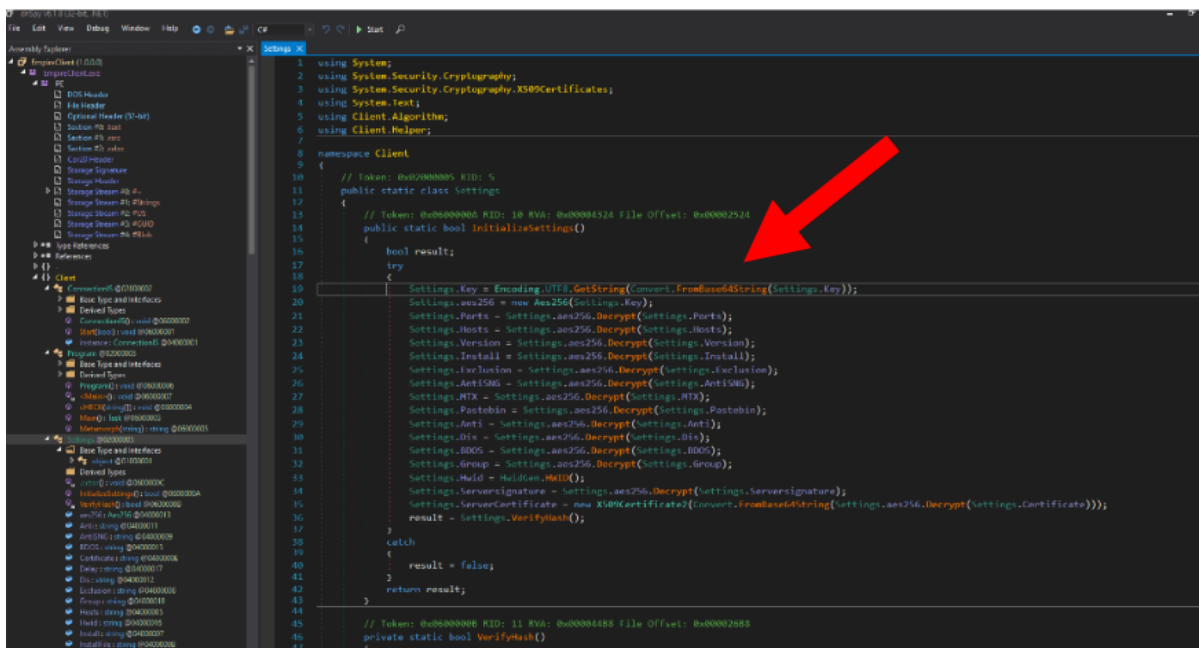
On line 28 of the unc.js file we locate a large base64 encoded blob, we copy and paste this into CyberChef and are able to confirm the presence of an executable, indicated by the files magic bytes of MZ.



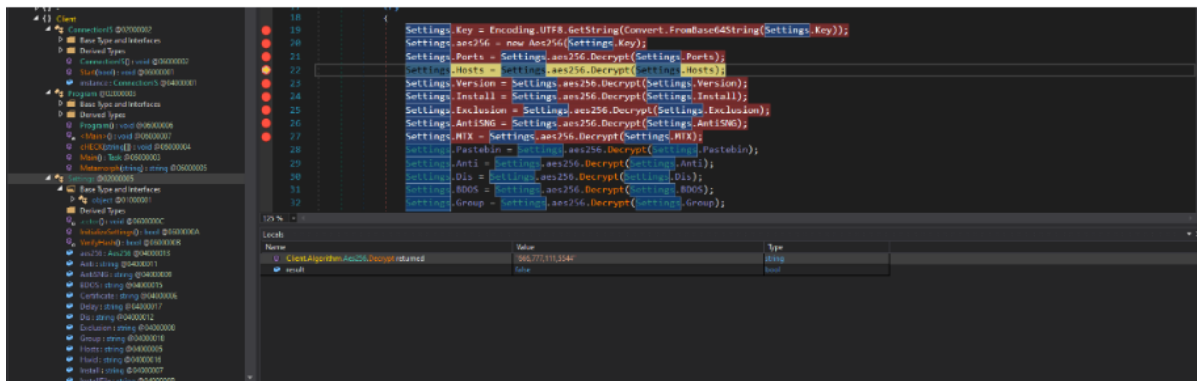
We proceed to download the file and then confirm the sha256 sum.

```
(sebh24@maroc) - [~/Desktop/einladen]
$ sha256sum download.exe
db84db8c5d76f6001d5503e8e4b16cdd3446d5535c45bbb0fca76cfec40f37cc  download.exe
```

Opening the EXE file in Detect it Easy, we confirm that it is a .NET binary. Upon decompiling the binary in DnSpy, we navigate to Client.Settings class in the source code and confirm there are numerous configurations that are being decrypted.

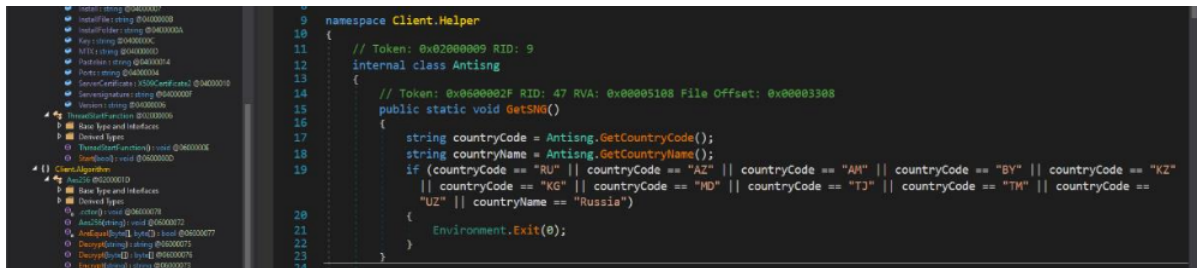


We set breakpoints within these statements and begin debugging, when we achieve a value of Ports. The ports detailed are 666, 777, 111, 5544.

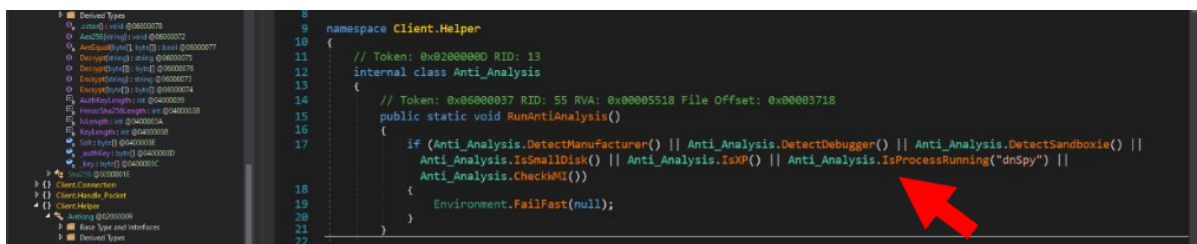


We additionally locate 2 functions related to Client.Helper.Antisng namespace: GetCountroCode() & GetCountryName(). The GetCountryName() function sends a HTTP GET request to the commonly used url ["http://ip-api.com/json/"](http://ip-api.com/json/) and collects the victim hosts country name and country code.

Within the same namespace, there is a function that confirms and performs a check if the country name is Russia or country code "RU". If it is, the malware will then terminate itself. This indicates a Russian actor performing the attack, highly likely state sponsored.



Continuing our analysis we are able to confirm that within the Client.Helper.Anti_Analysis namespace the malware uses the below code to check if the dnSpy debugger is running or not. This is defined an anti analysis technique.



Lets further breakdown the code detailed above:

1. **DetectManufacturer():** This method might check for specific attributes of the machine that could indicate it's part of a virtual machine (VM) or a specific manufacturer known for analysis environments. Many malware programs attempt to avoid running on VMs as they are commonly used for malware analysis.
2. **DetectDebugger():** The method checks if there is a debugger attached to the process. Debuggers like dnSpy are used by developers and analysts to inspect the execution of a program. If a debugger is detected, the malware may alter its behavior or terminate itself to prevent examination.
3. **DetectSandboxie():** This likely checks for the presence of Sandboxie, which is a sandbox-based isolation program used to run suspicious programs without risking the host system. Similar to VM detection, if the program detects that it's running in a sandbox, it might stop executing malicious activities.

4. **IsMalDisk():** This method name is not clear, but it could refer to checking whether the malware is running from a disk known to be malicious or perhaps checking for the presence of tools or environments used for malware analysis.
5. **IsXP():** This might be checking whether the operating system is Windows XP, which could be used by the malware to determine if it's running on an older and possibly less secure system, or it could be an attempt to identify a virtual machine that is often used for analysis as XP is an outdated system.
6. **IsProcessRunning("dnSpy"):** Specifically checks if the dnSpy debugger process is running. dnSpy is a popular .NET debugger and assembly editor used to debug .NET applications, and it's often used by analysts to decompile and debug malware.

The `RunAntiAnalysis` static method combines these checks with logical ORs, which means that if any of these checks return `true`, the following line `Environment.FailFast(null);` is executed. The `Environment.FailFast` method is a .NET Framework method that immediately terminates a program with a specified exit code, without executing any active try-finally blocks or finalizers. This would effectively stop the malware from running further and avoid its analysis if any anti-analysis conditions are met.

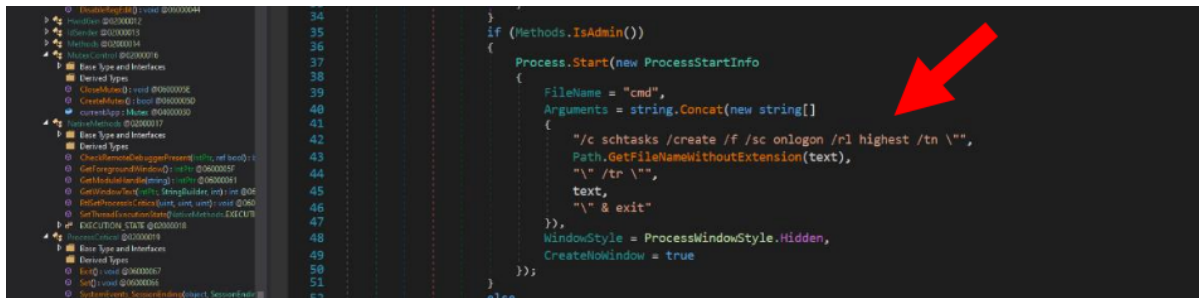
Delving into our `Client.Install.NormalStartup` namespace we find an interesting snippet related to our current users registry key, with a hardcoded value that looks slightly obfuscated on initial analysis.



Lets provide a detailed breakdown of the code within this namespace:

1. **Registry Access:** The code is using the `RegistryKey` class from the .NET Framework to access the Windows Registry. Specifically, it's targeting the `CurrentUser` hive, which pertains to the currently logged-in user.
2. **OpenSubKey:** It attempts to open a subkey for read/write access. The actual key name is obfuscated using a string reversal function which hints at a form of simple string obfuscation that's often used by malware to hide the registry keys it accesses or modifies.
3. **Check if the key exists:** It checks if the registry key exists (`if (registryKey != null)`). If it does exist, the code proceeds to modify it.
4. **SetValue:** The `SetValue` method is used to write a value into the registry. This could be used to store data that the malware needs to persist between system reboots, or to modify system behavior in some way. The value it's writing seems to be constructed from a path (`Path.GetFileNameWithoutExtension(text)`), followed by a `""` and more text. The `text` variable is not shown in the snippet, so it's unclear what it contains.
5. **Obfuscated Strings:** The snippet uses `Strings.StrReverse` to reverse the string literal, which means the actual key being accessed is reversed in this snippet. This is a common technique used in malware to make static analysis more difficult and to avoid detection based on string signatures. Taking this into account we are able to reverse the string to `HKCU\Software\Microsoft\Windows\CurrentVersion\Run\`
6. **Registry Key Permission Check:** It specifies `RegistryKeyPermissionCheck.ReadWriteSubTree`, which means the code requests read and write access not just to the specified key but to all its subkeys.

Further in the same namespace we find a secondary code snippet indicating scheduled task creation and the /rl command line flag setting the run level to the highest.



```
34 }
35 if (Methods.IsAdmin())
36 {
37     Process.Start(new ProcessStartInfo
38     {
39         FileName = "cmd",
40         Arguments = string.Concat(new string[]
41         {
42             "/c schtasks /create /f /sc onlogon /rl highest /tn \"",
43             Path.GetFileNameWithoutExtension(text),
44             "\" /tr \"",
45             text,
46             "\" & exit"
47         })),
48         WindowStyle = ProcessWindowStyle.Hidden,
49         CreateNoWindow = true
50     });
51 }
```

Questions

1. The victim visited a web page. The HTML file of the web page has been provided as 'downloader.html' sample file. The web page downloads ZIP file named 'Invitation_Farewell_DE_EMB.zip'. What is the SHA-256 hash of the ZIP file?
`5D4BF026FAD40979541EFD2419EC0B042C8CF83BC1A61CBCC069EFE0069CCD27`
2. The downloaded ZIP file contains a HTA file, which creates multiple files. One of those files is a signed file by Microsoft Corporation. In HTA file, which variable's value was the content of that signed file?
`msoev`
3. The threat actor was acting as an embassy of a country. Which country was that?
`Germany`
4. The malware communicated with a chatting platform domain. What is the domain name (inclusive of sub domain) the malware connects to?
`toyy.zulipchat.com`
5. How many DNS A records were found for that domain?
`6`
6. It seems like the chatting service was running on a very known cloud service using a FQDN, where the FQDN contains the IP address of the chatting domain in reverse format somehow. What is the FQDN?
`ec2-35-171-197-55.compute-1.amazonaws.com`
7. What was the parent PID (PPID) of the malware?
`4156`
8. What was the computer name of the victim computer?
`DESKTOP-088AN40`
9. What was the username of the victim computer?
`TWF`
10. How many times were the Windows Registry keys set with a data value?
`11`
11. Did the malicious mso.dll load by the malware executable successfully?
`yes`
12. The JavaScript file tries to write itself as a .bat file. What is the .bat file name (name+extension) it tries to write itself as?
`richpear.bat`

13. The JavaScript file contains a big text which is encoded as Base64. If you decode that Base64 text and write its content as an EXE file. What will be the SHA256 hash of the EXE?
- DB84DB8C5D76F6001D5503E8E4B16CDD3446D5535C45BBB0FCA76CFEC40F37CC
14. The malware contains a class Client.Settings which sets different configurations. It has a variable 'Ports' where the value is Base64 encoded. The value is decrypted using Aes256.Decrypt. After decryption, what will be its value (the decrypted value will be inside double quotation)?
- 666, 777, 111, 5544
15. The malware sends a HTTP request to a URI and checks the country code or country name of the victim machine. To which URI does the malware sends request for this?
- http://ip-api.com/json/
16. After getting the country code or country name of the victim machine, the malware checks some country codes and a country name. In case of the country name, if the name is matched with the victim machine's country name, the malware terminates itself. What is the country name it checks with the victim system?
- Russia
17. As an anti-debugging functionality, the malware checks if there is any process running where the process name is a debugger. What is the debugger name it tries to check if that's running?
- dnSpy
18. For persistence, the malware writes a Registry key where the registry key is hardcoded in the malware in reversed format. What is the registry key after reversing?
- HKCU\Software\Microsoft\Windows\CurrentVersion\Run\
19. The malware sets a scheduled task. What is the Run Level for the scheduled task/job it sets?
- highest